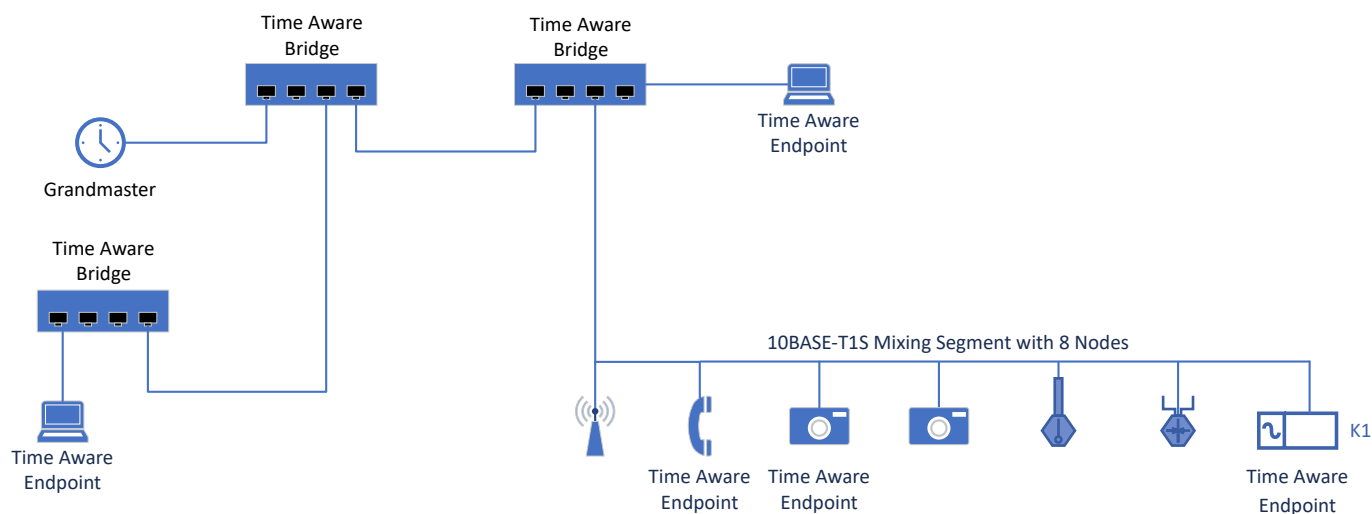


Overview

The LAN8650/1 contains hardware features to enable the development of time-aware systems using 10BASE-T1S multidrop mixing segments. This application note explains how to take advantage of these time synchronization features.



General Terms

Table 1. General Terms

Term	Description
1PPS	One Pulse Per Second
10BASE-T1S	10 Mbit/s Ethernet over short-reach single pair of conductors, IEEE Std 802.3 Clause 147
AVB	Audio Video Bridging
gPTP	Generalized Precision Time Protocol, IEEE Std 802.1AS
PI	Proportional-Integral
PID	Proportional-Integral-Derivative
PLCA	Physical Layer Collision Avoidance, IEEE Std 802.3 Clause 148
PTP	Precision Time Protocol, IEEE Std 1588
SFD	Start-of-Frame Delimiter

Table of Contents

Overview.....	1
General Terms.....	2
1. Introduction.....	4
2. Using PTP Messages for Time Synchronization	6
3. LAN8650/1 Support for Time Synchronization.....	12
4. Implementing a Simple Clock Follower in the LAN8650/1.....	15
5. Conclusion.....	19
6. References.....	20
7. Revision History.....	21
Microchip Information.....	22
Trademarks.....	22
Legal Notice.....	22
Microchip Devices Code Protection Feature.....	22

1. Introduction

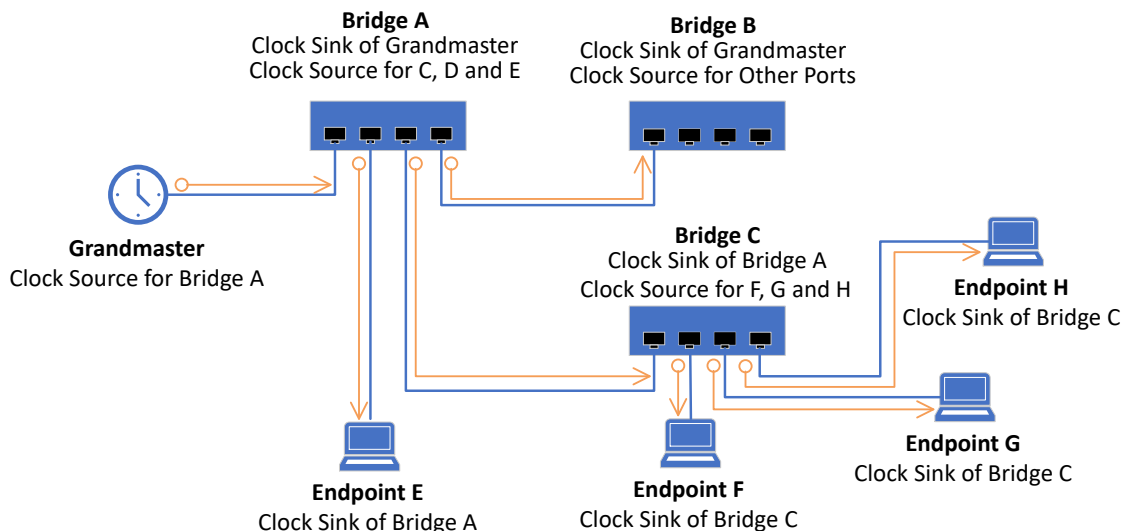
10BASE-T1S networks enable system designers to expand standard Ethernet networks to include low-bandwidth devices on multidrop networks. These devices were previously connected via a variety of interfaces, such as CAN, LIN and RS-485, each of which requires specialized gateways to bring their data into the Ethernet ecosystem. An advantage of standardization is the ability to use existing software stacks which provide additional features that are built upon Ethernet. One of these key features available for Ethernet based systems is support for time aware networks, which enables clock synchronization and precision timestamps across multiple network segments. In particular, IEEE Std 1588 describes version 2 of the Precision Time Protocol, PTPv2, for precision clock synchronization in a time-aware system.

IEEE Std 1588 can be implemented in a variety of profiles which describe sets of standards including combinations of optional features optimized for different end applications. One common profile described in IEEE Std 802.1AS specifies a generalized Precision Time Protocol (gPTP) profile, which is also the profile used by Ethernet Audio/Video Bridging (AVB). The generalized precision time protocol is often used synonymously with PTP and with time sensitive networking, but it is only a subset of possible options.

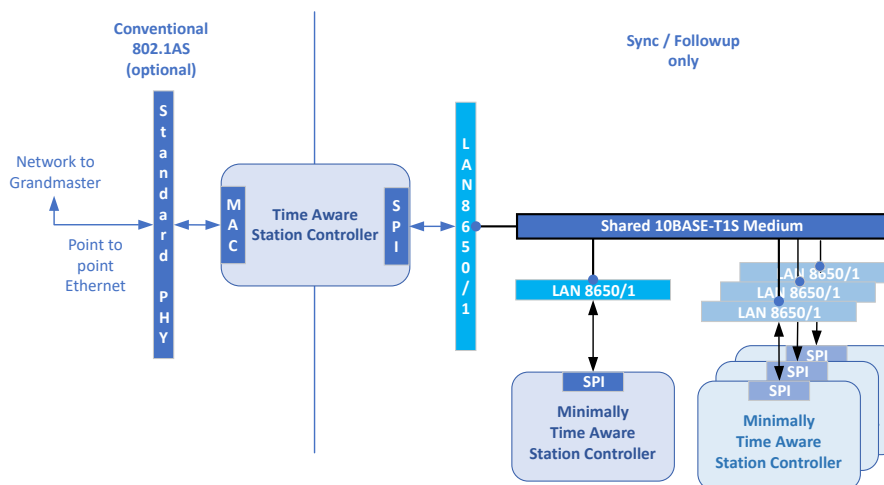
A time-aware system consists of one or more network segments containing *endpoints* connected by *bridges*. The endpoint with the best clock is known as the *grandmaster*; gPTP allows the grandmaster's clock to be distributed across the network. The grandmaster is connected to a time aware bridge via a point-to-point connection. The bridge can connect to other time sensitive bridges or connect with time sensitive endpoints. In [Figure 1-1](#), the grandmaster is connected to bridge A. Bridge A will synchronize its clock directly to the grandmaster. Bridge A is connected to bridges B and C as well as endpoint E and all 3 synchronize their clocks to bridge A. Because bridge A is synchronized to the grandmaster, B, C and E are also synchronized to the grandmaster. Any devices connected to ports of Bridge B will synchronize their clocks to the clock of Bridge B. Any devices connected to Bridge C, in this case F, G and H, will synchronize their clocks to Bridge C. This method is called a peer-to-peer synchronization scheme. IEEE Std 802.1AS ensures synchronization accuracy of 1 ms or better between the grandmaster and endpoints separated by up to six bridges using this synchronization method.

In any time-aware network, any time-aware bridge will have one port which the bridge will use to synchronize its wall clock. This port is connected to a *clock source*, which can be the grandmaster or another time-aware bridge. The PTP protocol is used to convey the timestamps necessary for synchronization. For bridge A in [Figure 1-1](#), the clock source is the grandmaster, while for bridges B and C, the clock source is bridge A. Clock sources can also be called *clock transmitters*. When distinguishing a single link of this chain, a clock source may be called a *local clock source*, as in bridge A is the local clock source for bridge C or bridge C is the local clock source for endpoint F.

A time-aware bridge synchronizes its clock from a clock source on one port, and acts as a clock source for its other ports. Each output port can be connected to either endpoints or other bridges. The bridge will use its wall clock, calculates delays from both the network and internal sources and uses that calculation to generate the correct timestamps and correction fields required for the PTP protocol. A device that is not a clock source can be called a *clock sink* or a *clock receiver*. Bridges are clock sinks on the port that is connected to the clock source, but clock sources on other ports. The orange arrows in [Figure 1-1](#) use a circle to indicate the clock source for a particular connection and an arrow to indicate the clock sink.

Figure 1-1. Example Time Aware Network

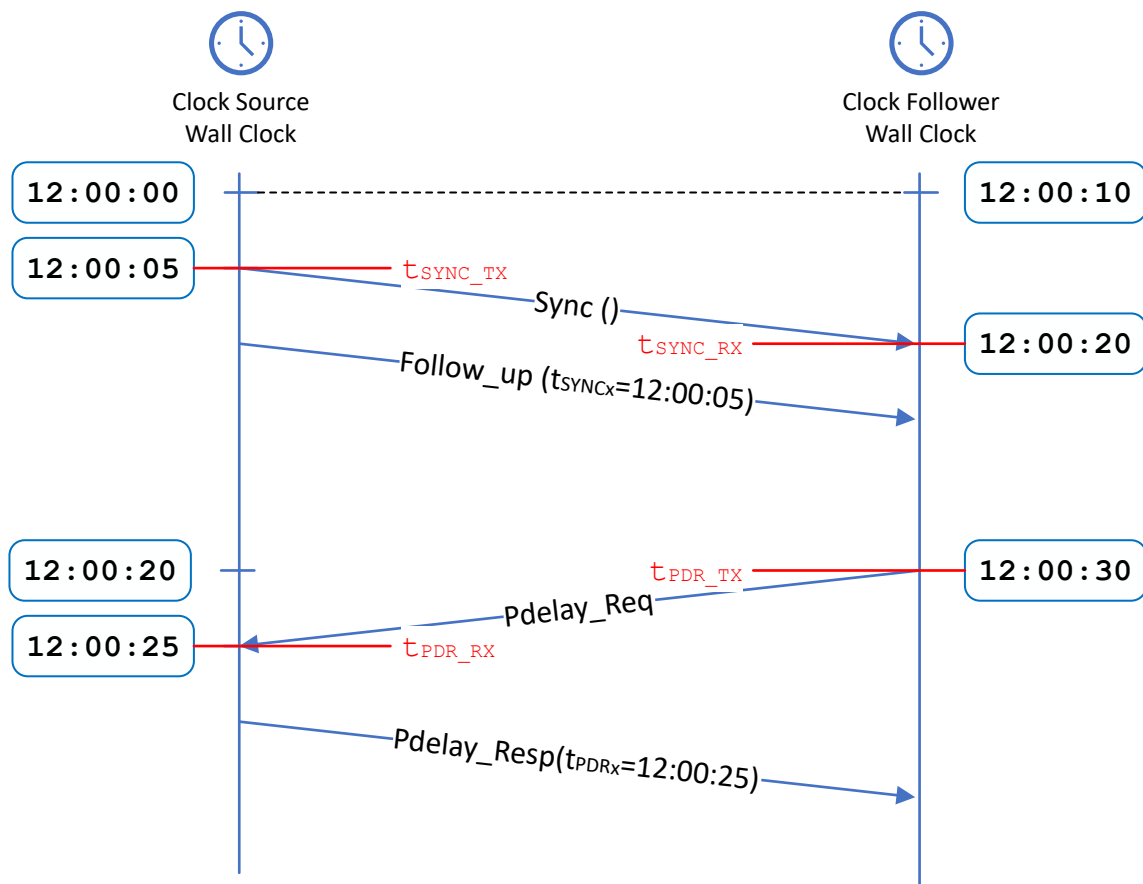
Full implementation of IEEE 802.1AS requires software implementation of a large number of protocols and often takes advantage of special hardware features like priority queuing. Controllers which support these features are more expensive than is otherwise needed for a simple sensor, actuator or LED driver. As an alternative, [Figure 1-2](#) shows a very simple example of a network containing nodes based on the LAN8650/1, where each device's local wall clock can be synchronized to provide timestamping for local events. Here, the time aware bridge is shown in three parts: the time aware station controller and its two Ethernet interfaces. The bridge's wall clock acts as the local clock source for the mixing segment: it provides the messages that enable the minimally time aware devices on the mixing segment to synchronize their wall clocks. The time aware station controller can optionally synchronize its wall clock from an external grandmaster using IEEE 802.1AS, in which case this controller requires hardware support for time sensitive networking, such as MAC timestamping and priority queues. The minimally time aware systems in the figure are simple microcontrollers. They are assumed to have slower clock speeds and small memories, and will have no hardware support design to assist with time sensitive networking. This application note will show how these small devices can still have clocks synchronized to the local clock source, including description of necessary software.

Figure 1-2. Example of a Synchronized Mixing Segment

2. Using PTP Messages for Time Synchronization

Any time aware network segment will contain one clock source, either a bridge or the grandmaster. The clock source uses PTP messages to distribute its clock to its clock followers, as shown in Figure 2-1. The clock source will periodically send a PTP *Sync* message, which contains the timestamp when the packet was assembled. Since there will be some delay between assembling of the packet and its transmission as the packet moves through various software layers and hardware buffers, the timestamp contained in the assembled *Sync* packet is not the exact transmission time. All time aware devices contain hardware which can capture the required timestamps. The timestamp for the *Sync* packet is then provided later in a *Follow_up* packet. In some cases, the MAC is able to update the original timestamp in the *Sync* packet to the correct value while it is going through the MAC; in this case no *Follow_up* packet is needed. To indicate that a *Followup_up* packet will be sent, the *twoStepFlag* bit will be set in the *flagField* of the *Sync* packet.

Figure 2-1. PTP Message Timing Example



When the clock follower detects a *Sync* packet, it will capture a timestamp based on its own wallclock. Even on a simple local network segment, there will always be some delay between the clock source and clock follower. This delay includes the propagation time for the signal to traverse the network, as well as any delays caused by electronic components or buffers in hardware. To determine the total delay, the clock follower requires more information. To acquire this information, PTP uses two more message types with corresponding timestamps. The follower sends a *Pdelay_request* message to the clock source and captures a timestamp when the message is transmitted. The clock source will return a *Pdelay_response* message which contains the clock source timestamp taken when the *Pdelay_Request* is received.

At the end of this set of messages, the follower has received two timestamps from the clock source: $t_{\text{SYNC_TX}}$, the time that the *Sync* message was sent and $t_{\text{PDR_RX}}$, which is when the clock source received the *PDelay_Req* packet. The follower has also captured two timestamps based on its own local clock, $t_{\text{SYNC_RX}}$, when it received the *Sync* packet and $t_{\text{PDR_TX}}$, when it transmitted the *PDelay_Req*. Assuming a symmetric and constant delay, it is possible to calculate the propagation delay and the error in the local clock from these four timestamps using the following formulas:

Equation 2-1.

$$t_{PD} = ((t_{PDR_RX} - t_{SYNC_TX}) - (t_{PDR_TX} - t_{SYNC_RX})) / 2$$

Equation 2-2.

$$\text{Clock Error} = (t_{SYNC_RX} - t_{SYNC_TX}) - t_{PD}$$

As an example, using the values in [Figure 2-1](#)

$$t_{PD} = (20s - 10s) / 2 ((12:00:25 - 12:00:05) - (12:00:30 - 12:00:20)) / 2 = (20s - 10s) / 2 = 5s$$

$$\text{Clock Error} = (12:00:20 - 12:00:05) - 5s = 15s - 5s = 10s$$

Modifications Needed to Support Multidrop and PLCA

At the time of publication, networking standards do not yet consider multidrop network segments to be “time-aware” and standard software implementations do not provide full support for multidrop networks. The *PDelay* messages are currently defined as broadcast messages. If multiple devices on a multidrop segment send *PDelay_req* messages and each uses the broadcast address, it will be unclear which device sent the message and which device needs to return the timestamp response. If *PDelay* messages are used, software must be customized to support multiple nodes generating these messages, and the clock source must capture multiple timestamps and return each to the proper requesting device. These exact modifications are software dependent and are beyond the scope of this application note.

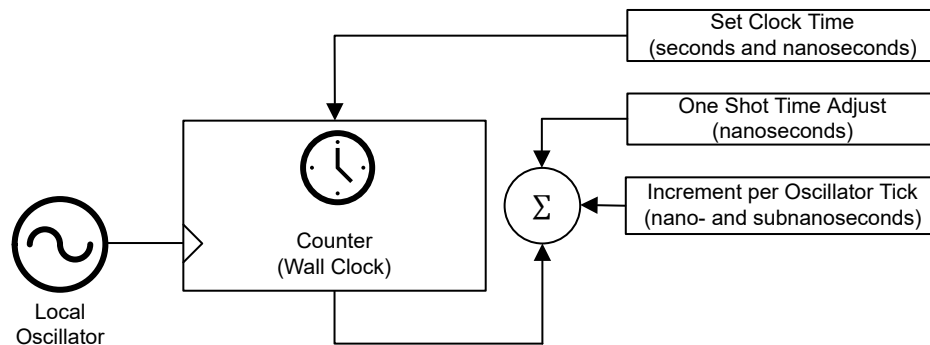
The standards also assume that the timestamp is taken when a first nibble following the Start-of-Frame Delimiter (SFD) leaves the MAC, but the elastic buffer of PLCA causes a small bounded but variable delay through the PHY. To achieve a constant delay, transmit timestamps need to be taken in the PHY; as long as the propagation delay remains constant, it does not matter exactly where it is taken. To achieve a symmetric delay for transmit and receive, receive timestamps also need to be taken at the PHY. The LAN8650/1 includes hardware support to allow timestamps to be taken when the packet is actually being transmitted. Based on this hardware, here are two propagation delay options that could be implemented:

- Determine (by measurement, simulation, or theoretical calculation) the peer delay between the clock source and every other node on the mixing segment and use this as a fixed value directly in [Equation 2-2](#). If the propagation delay is significantly smaller than the required timestamp precision, it may even be ignored and compensated for by the error correction. This method only uses *Sync* messages, which are only transmitted from the clock source and can continue to use broadcast address.
 - Timestamp that must be captured on the clock source - *Sync* transmit
 - Timestamp that must be captured on the clock follower- *Sync* receive
- Determine propagation delay using [Equation 2-1](#) after modifying the software to work around the broadcast issue for *PDelay_Req* and *PDelay_Resp*.
 - Timestamps that must be captured on the clock source - *Sync* transmit, *PDelay_Req* receive
 - Timestamp that must be captured on the clock follower - *Sync* receive, *PDelay_Req* transmit

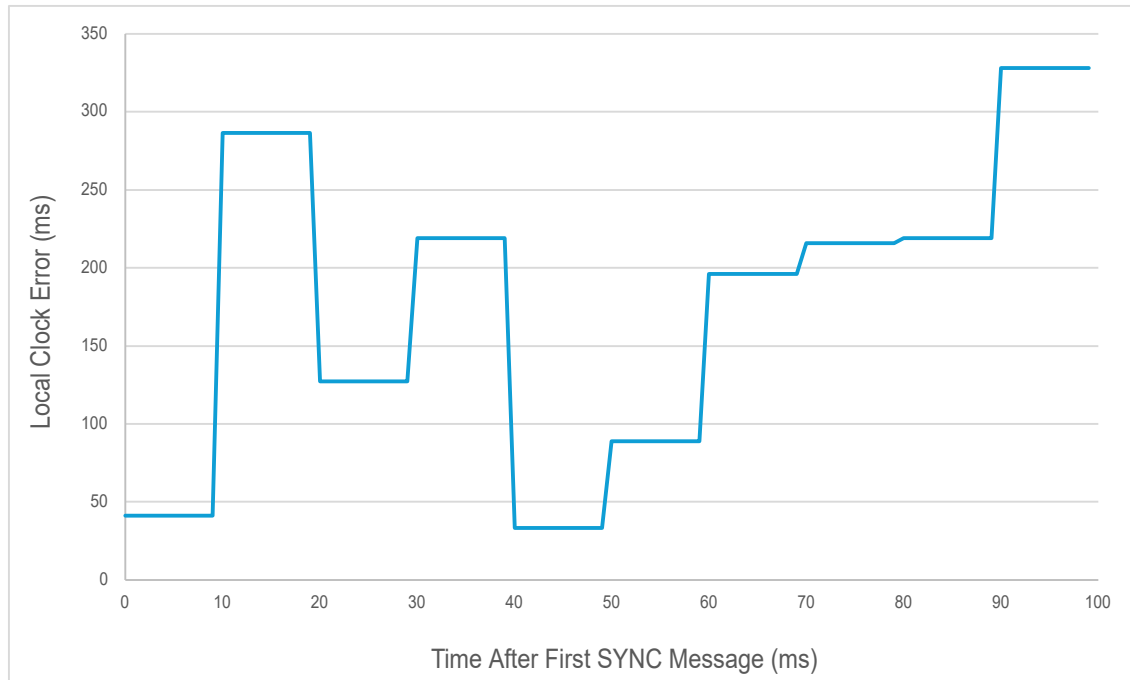
Synchronizing the Wall Clock of the Clock Follower

Wall clocks are implemented as counters, as shown in [Figure 2-2](#). Its time can be set or changed by writing to a set of registers. After a clock has been set, it continuously updates based on a local oscillator. The amount that a wall clock increments per oscillator tick is also controlled by one or more increment registers. These increment registers enable control down to sub-nanosecond resolution; this precision is needed to compensate for the frequency drift of the local oscillator. Wall clocks often also have an additional register that allows a one time addition or subtraction of an adjustment to the clock value; this register would allow a clock that is a few nanoseconds fast or slow to be corrected in a single tick of the local oscillator.

Figure 2-2. Block Diagram of a Wall Clock Plus Oscillator

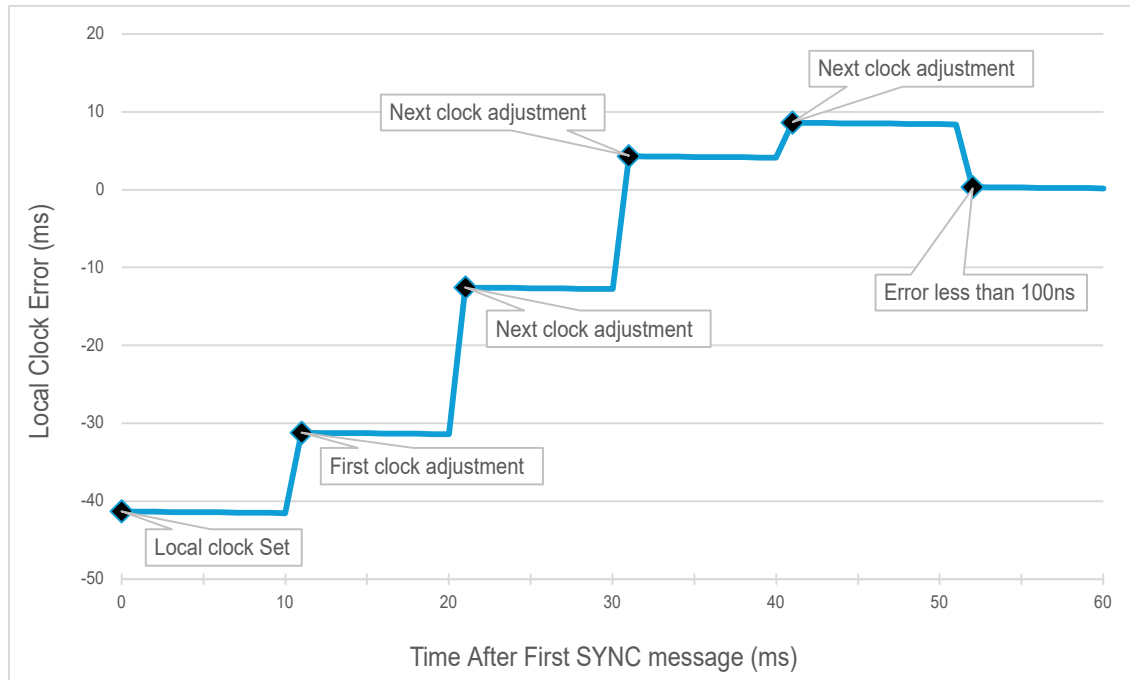


Once the error between the follower's wallclock and the clock source is determined from PTP messages, the wall clock needs to be updated to drive the error to zero. A simple way to do this would be to update the wall clock once per *Sync* message to eliminate the clock error. While this method is straightforward, it results in an error that is based on the time required to process the inbound messages and write the update to the wall clock. [Figure 2-3](#) shows the clock error over the first second from a simulation using random timing for the clock updates based on a realistic model of the LAN8650/1. Because the register updates are done over the same Serial Peripheral Interface (SPI) used to transmit Ethernet packets, there is a significant variation in write times, which results in the large variation in clock error.

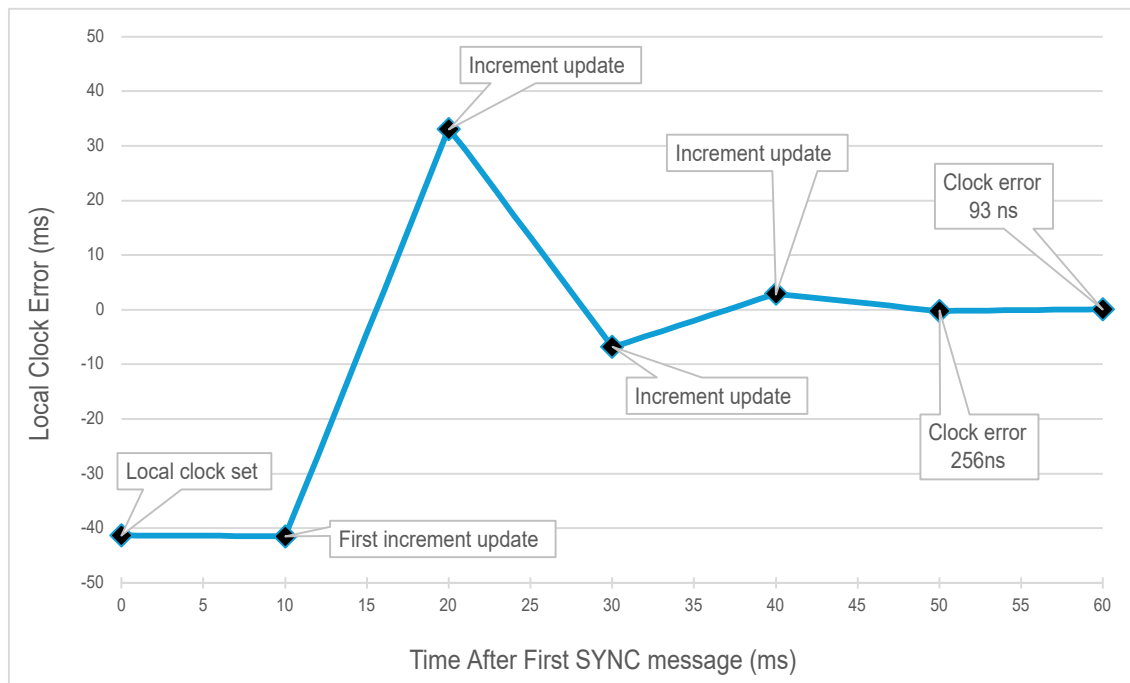
Figure 2-3. Local Clock Error when Updating the Clock Time

Even in devices that do not use SPI, updating the clock value alone does not produce reliable synchronization. No matter how accurate the timestamp and the propagation delay on the transmission line can be evaluated, the local clock will not be completely synchronized with the clock source. This is due to the delay between obtaining the last timestamp and updating the local clock, including the time required to calculate the updated value. Any time variation in this procedure will further add jitter to the clock error. As a result, setting clock time directly is normally only done once to set the initial clock value.

Another method to adjust the clock is to adjust once per *Sync* message. A simulated example of this is shown in [Figure 2-4](#). Using this method will cause an immediate shift in the wall clock and may result in timestamps being slightly out of sequence but, for systems can tolerate such clock jitter, this easy-to-implement method may be all that is required.

Figure 2-4. Local Clock Error Using only Phase Adjust

Instead of trying to update the clock each time a *Sync* message is received, most synchronization algorithms modify the registers which contain the amount of time the wall clock increments per tick of the local oscillator. The simulation results in [Figure 2-5](#), which use the same update delay jitter as above, show that modifying this increment once per *Sync* message enables the local wall clock to be quickly synchronized to the clock source. In this simulation, the value that the increment is updated is based on a filtered value of the error.

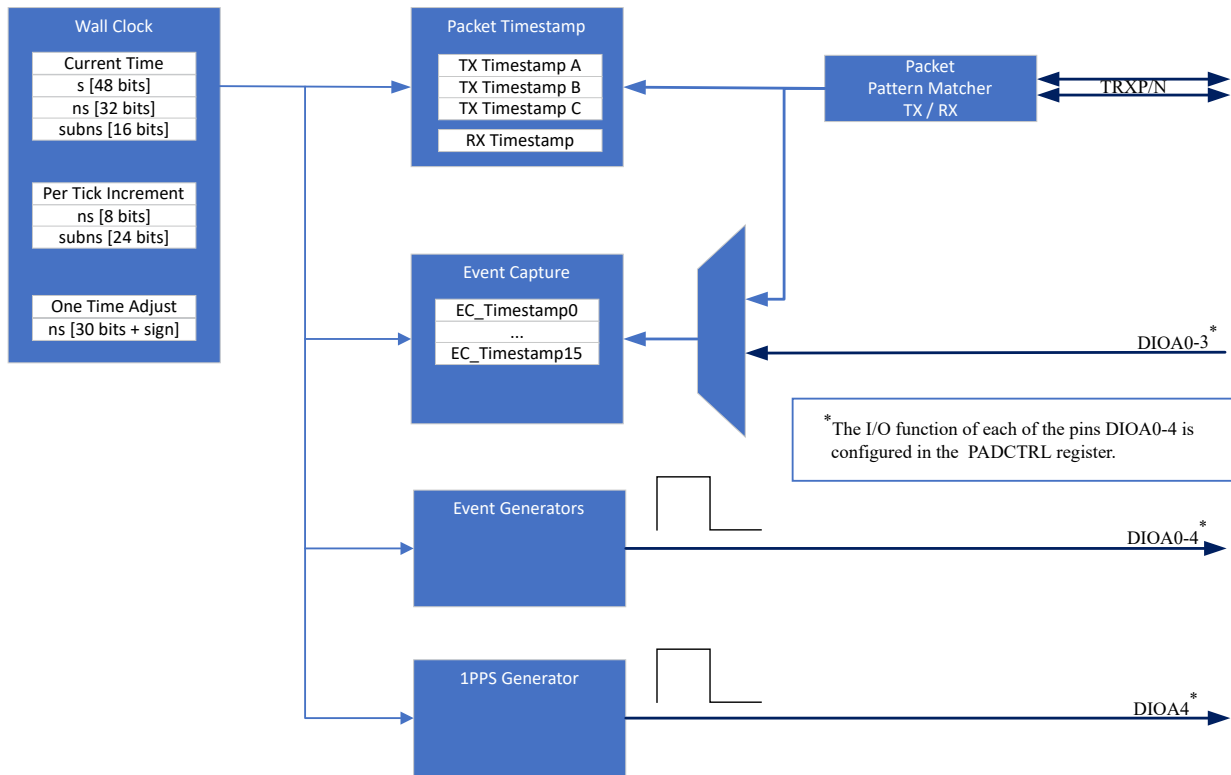
Figure 2-5. Local Clock Error when Updating the Increment

Algorithms designed to synchronize and maintain clocks vary greatly. Simple ones, used in systems with minimal clock skew, require only filtered values of the clock error. Other algorithms will also analyze the *rate ratio*, that is, the time difference between successive *Sync* message timestamps on the source and local clocks. More complicated clock servo algorithms add Proportional-Integral (PI) or Proportional-Integral-Derivative (PID) control loops based on both of clock error and rate ratio, and use a combination of adjusting the increment when error is small with one-time clock adjustments when the error is larger. Most algorithms use all three methods of adjusting the clock; the exact method used depends on recent error history.

3. LAN8650/1 Support for Time Synchronization

The LAN8650/1 includes hardware features to enable clock synchronization and to use the synchronized clock to synchronize external signals and capture timestamps from external events. Figure 3-1 provides an overview of these hardware features. This application note presents a simple application concept; as a result, not all of the time synchronization features listed in the datasheet are used.

Figure 3-1. Time Synchronization Features of the LAN8650/1



Wall Clock

The wall clock or local clock is a set of internal counter registers which are used by software to synchronize the device to an external clock source. The wall clock provides the timestamps used for inbound and outbound packets in the MAC and in event timestamp unit. It is also used by the event generator to generate synchronized signals.

In the the LAN8650/1 the wall clock is a 94-bit counter where:

- The 48 upper bits [93:46] represent seconds.
- The 30 lower bits [45:16] represent nanoseconds. This field resets to 0 at the end of each second.
- The lowest 16 bits [15:0] of the timer count sub-nanoseconds. This field resets to 0 at the end of each nanosecond. It is not accessible by register read or write.

Software can directly update the seconds and nanoseconds portions of the wall clock by writing to registers. Most implementations will directly modify the wall clock once, at the start of clock synchronization.

The clock will increase in value every tick of the local oscillator by the value contained in increment registers. For this device, the value is nominally set to 40 ns corresponding to the specified 25 MHz input clock.

Once the initial value is written, software then tunes the wall clock so that it synchronizes to the clock source. It does not change the wall clock time directly, but uses one of two means to modify the behavior. It can either modify the increment for each tick of the local clock, down to fractions of a nanosecond where needed, or it can perform a single adjustment to increment or decrement the wall clock. The exact method used can be determined by the designer of the synchronization algorithm.

Event capture and event generation use all 48 seconds bits and all 30 nanoseconds bits. Packet timestamps accessed using the built-in features of the SPI protocol will use the least significant 2 or 32 seconds bits depending on the selected timestamp format.

Packet Pattern Matcher

The LAN8650/1 includes a packet pattern matcher that enables accurate timestamping. When using the round trip delay calculation of [Equation 2-1](#), it is assumed that the delay is equal in both directions. As described above, the PLCA elastic buffer contributes a variable length delay during transmission. In the LAN8650/1, the transmit pattern matcher signals the timestamp unit that the end of the SFD has been transmitted by the PHY, so that the time stamp is captured at this time, instead of using the time the SFD leaves the MAC. This ensures a consistent internal delay when transmitting packets. To ensure that the receive timestamps are made with a similar delay, the receive packet pattern matcher is used to trigger the timestamping at the end of the SFD in receive packets.

Packet Timestamping

When configured to do so, the packet timestamp unit captures a timestamp when signaled by the pattern matcher. The SPI MAC-PHY protocol enables configuration of timestamps and provides them to the controller as follows:

- Receive packet timestamps:
 - Receive packet timestamps for all incoming packets are enabled by setting the Frame Timestamp Enable (FTSE) bit of OA_CONFIG0.
 - The Receive Timestamp Added (RTSA) bit will be set in the footer along with the Start Valid (SV) bit.
 - The timestamp will be prepended to the packet data and delivered in the first data block of the packet.
 - The Receive Timestamp Parity bit (RTSP) of the footer will be '1' if the timestamp parity is odd, else it will be '0'.
 - The timestamp width (32 or 62 bits) is selected using the Frame Timestamp Select (FTSS) bit in the Configuration 0 Register (OA_CONFIG0).
- Transmit packet timestamps:
 - A timestamp is requested for a particular packet by setting the Timestamp Capture (TSC) field of the data header of the first block of data transmitted for that packet.
 - The TSC is a two-bit field, which indicates which of the Transmit Timestamp Capture High/Low register pairs should contain the timestamp.
 - When the frame is transmitted, the timestamp will be captured, placed into desired Transmit Timestamp Capture register pair, and set the appropriate Transmit Timestamp Capture Available (TTSCAA, TTSCAB, TTSCAC) status bit in the Status 0 (OA_STATUS0) register.
 - A successful timestamp capture can be used to trigger an interrupt, if desired.
 - The timestamp capture registers can be read like any other registers on the device, using SPI control transactions.

**Important:**

The LAN8650/1 is not configured for correct timestamp capture after reset, but the default configuration set by all Microchip drivers ensures proper configuration. See the LAN8650/1 Configuration Application Note for details, especially if writing your own driver. Software must additionally request transmit and receive timestamps via the SPI protocol and process the resulting timestamps.

Event Timestamping

The event capture unit can simultaneously monitor up to four events and capture the wall clock time at which the configured event occurs. Each event has its own configuration register, which can select the source from DIOA0-3 or either the transmit or receive pattern matcher. When DIO pins are used, they must also be configured as event capture inputs in the PADCTRL register.

To enable efficient use of SPI bandwidth, all resulting status and timestamps are available in a group of sequentially accessed registers. This allows for use of the auto-increment feature of the SPI control protocol to burst read all of the required registers in a single message.

Synchronized Event Generation

There are four configurable event generators, each of which can generate a single edge or configurable width pulse from the same wall clock used for timestamp generation. The generated signals can be routed to pins DIOA0-3 so they can be used to synchronize external devices. The routing is controlled by the PADCTRL register.

1 Pulse Per Second

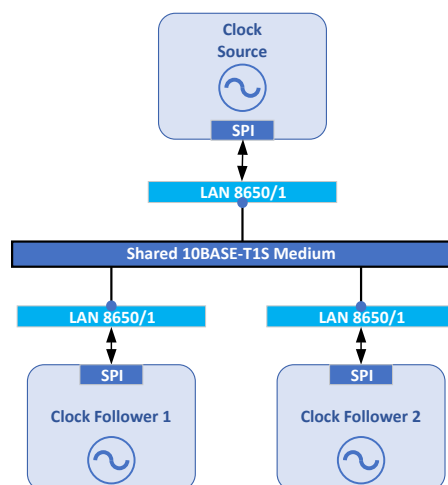
The device can provide a one pulse-per-second (1PPS) clock synchronized to the wall clock. This signal can be provided on pin DIOA4. The pulse width can be configured between 640 ns and 20.48 μ s. The 1PPS signal is output on pin DIOA4 when configured in the PADCTRL register. A 1PPS signal is often used to monitor the synchronization of the clock or as a reference for other PLLs.

4. Implementing a Simple Clock Follower in the LAN8650/1

Figure 4-1 shows a block diagram for a synchronized mixing segment based on the LAN8650/1. The clock source could be a simple node that sets the standard for the network but does not support the full IEEE 802.1AS standard. In this case, it must be able to generate and send *Sync* and *Follow_up* messages with correct timestamps to the shared medium. It could also be a more powerful node that connects to an external grandmaster via a point-to-point network that is not shown. Such a node will support the full IEEE 802.1AS standard, although it may need modification to its protocol stacks so that it sends only supported traffic onto the mixing segment.

The clock followers are processors that do not contain MACs and so will use SPI to connect to the LAN8650/1. They will use the wall clock and packet timestamping features to synchronize clocks using only received *Sync* and *Follow_up* messages. The quality of the clock can be monitored using the 1PPS output function. The event generator can be used to provide events synchronized to the wall clock and event timestamping can be used to timestamp external events.

Figure 4-1. Simple Mixing Segment



Software Configuration for a Clock Follower

Any processor using a LAN8650/1 requires a driver to interface to the device, such as the LAN865x Driver included in MPLAB® Harmony v3. This driver will process both inbound and outbound Ethernet packets as well as control and monitor the device.

The driver will begin with configuring the device as documented in the LAN8650/1 Configuration Application Note. This configuration includes the following settings that are needed for a clock follower:

- Configure the pattern matcher so that timestamps are generated following the Start-of-Frame delimiter on all receive packets
 - Set the receive match mask to 0xFFFFFFFF by writing to the Receive Match Mask (High) (RXMMSKH) and Receive Match Mask (Low) (RXMMSHL) registers
 - Set the Receive Match Location (RXMLOC) to 0
- Configure the wall clock increment register to the nominal value of 40 ns for each tick of the 25 MHz oscillator
 - Set the TSU_Timer Increment Register (MAC_TI) to 0x28

In addition, clock followers will require the application to configure and enable the inclusion of all timestamps for received packets as described in the serial interface specification.

- Set the FTSE bit in the OA_CONFIG0 register to enable timestamping.
- Set the FTSS bit in the OA_CONFIG0 register to '1' for 64-bit timestamps or '0' for 32-bit timestamps

PTP Message Handling

Once the device configuration is complete, clock follower software needs to add handling for the PTP packets *Sync* and *Follow_up*

- The *Sync* message contains a sequence number. This sequence number is repeated in the *Follow-up* packet. These numbers should be checked to ensure the timestamp corresponds to the correct *Sync* message.
- The timestamp $t_{\text{SYNC}_{\text{TX}}}$ is found directly inside the *Follow_up* packet. In the example, the TC6 driver delivers the full Ethernet packet and the example software extracts it.
- The timestamp $t_{\text{SYNC}_{\text{RX}}}$ is captured by the packet timestamp unit and is provided to the controller as described in the SPI MAC-PHY protocol. The LAN865x Driver copies timestamps for all PTP packets and passes them as arguments to the *handlePtp* function.

Sync packets are typically sent at 8 or 16 packets per second. In many systems, the clocks are fully synchronized after less than 20 *Sync* messages

Synchronizing the Clock

After processing the PTP messages, the timestamps are used by the clock synchronization algorithm. This algorithm implements a software phase-locked loop that controls the local wall clock, such as one shown in [Figure 4-2](#). The algorithm will process the input timestamps and determine the difference between the source wall clock and the local wall clock, which is the time error (sometimes called phase error). Most algorithms also track the time between *Sync* timestamps on both source and follower and use the resulting rate ratio to track frequency drift. Synchronization algorithms can vary from simple filters for networks that have low traffic loads and therefore minimal jitter to algorithms containing PI or PID controllers for noisier networks.

This method requires an estimate of the propagation delay between the clock follower and each node. This can be done by measurement or simulation of the mixing segment. Note that if the propagation delay is small, that is, if the mixing segment is short, then the clock servo algorithm may choose to omit the propagation delay. As a rule of thumb, a propagation delay corresponding to one meter of twisted pair cable is approximately 5 ns.

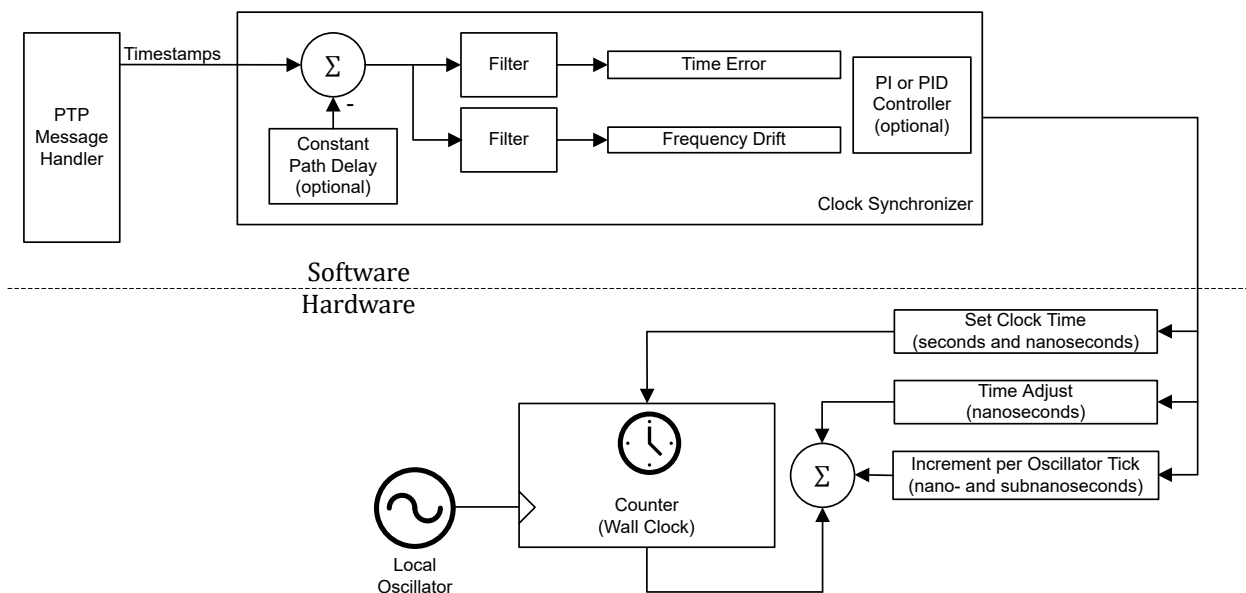
Clock servo algorithms often have multiple states as they align and maintain the clocks over time. A typical algorithm implements the following states:

1. Initialization: At start-up, or if the input clock source changes, the clocks will vary greatly. In this phase, the clock servo algorithm will normally directly write the values of the wall clock. This is done by writing to the registers TSU Timer Seconds High Register (MAC_TSH), TSU Timer Seconds Low Register (MAC_TSL) and TSU Timer Nanoseconds Register (MAC_TSN). This is normally done once, then the algorithm will move to the Unlocked state.

2. Unlocked: After initialization, the clock needs further alignment before it is ready to be used. This state can also occur when a previously locked clock is now experiencing enough jitter that it causes errors that cannot be handled in the locked states. In this state, the algorithm will use large adjustments of the wall clock by writing to the Timer Adjust (MAC_TA) register. When the clock error is within limits, the stage will change to Locked (Coarse).
3. Locked (Coarse): In this state, the clock error is small enough so that the clock frequencies are very closely synchronized, but the phase error is still large. Depending on the algorithm, adjustments on the order of a few nanoseconds may be made via the MAC_TA register, or adjustments may be limited to modifying the increment with changes to the MAC_TI and MAC_TISUBNS registers.
4. Locked (Fine). This state is reached when the clock error is within acceptable limits, the frequencies are well matched and wall clock has only a small phase error, which can be corrected with either small changes to the increment registers or time adjust register. The error limit as well as the correction method are dependent on the total noise and jitter of the system, including the Grandmaster clock.

The quality of synchronization can be examined by comparing a 1PPS output from the clock follower to that of the clock source. In the example, the 1PPS output is enabled the first time that the software enters the FINE state.

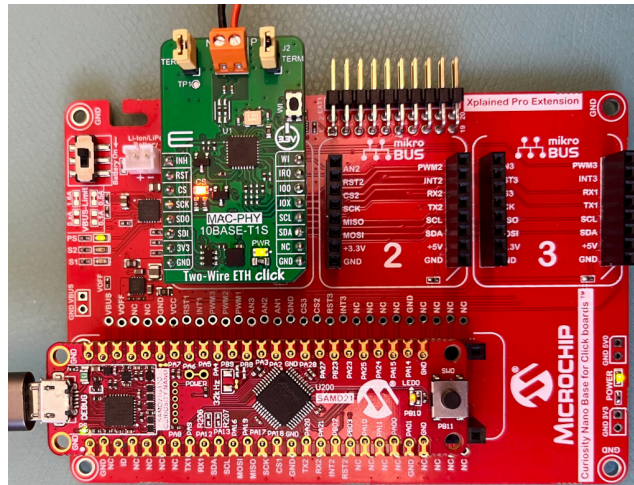
Figure 4-2. Clock Synchronization Software and Hardware



Results

An example of the code for both a PTP Grandmaster and PTP follower can be found online under github.com/MicrochipTech/LAN865x-TimeSync.

The example algorithm was tested in a system under laboratory conditions using a LAN8650/1 as the Grandmaster clock source. The local follower shown in Figure 4-3 was built with the SAM D21 Curiosity Nano Evaluation Kit connected to a Two-Wire ETH Click board, containing a LAN8650/1 using the Curiosity Nano Base for Click Boards. It was developed using MPLAB Harmony v3 framework, so the software can run on many Microchip evaluation boards. It uses 64-bit timestamps and the clock source provided eight SYNC messages per second. Essentially the Grandmaster and the follower nodes are running on the same hardware. The boards are connected by about 50 cm of unshielded twisted pair cables.

Figure 4-3. Example Evaluation Hardware

The Grandmaster device includes features such as the ability to delay the SYNC and FOLLOWUP messages by a predetermined, variable amount for robustness testing of the PTP follower and the ability to transmit iPerf messages between SYNC messages. The Grandmaster device follows the correct method for accessing registers on the LAN8650/1 for transmitting a timestamp.

The PTP follower device contains only simple algorithms to synchronize its internal clock. Before synchronizing the time, the PTP follower uses the first few SYNC and FOLLOWUP messages to guess the difference between its own clock frequency and the clock frequency of the Grandmaster. It adjusts this once before beginning the clock synchronization algorithm. During this algorithm, the follower only uses simple FIR filters to adjust its own internal time. This makes the algorithm extremely simple and able to be used in nearly any microcontroller. The different states described above are reached when the time difference between the internal times reach various thresholds. Time adjustments are done exclusively through the time adjust register, MAC_TA.

Comparing the 1PPS output of this node to the clock source shows the follower clock to have a maximum time difference of 100 ns, with an 8 ns mean and standard deviation of 25 ns. This performance is acceptable for many applications. If more precision is required, or the end system has more jitter, the references section contains additional information for more complex clock synchronization algorithms.

5. Conclusion

The LAN8650/1 includes several hardware features to enable time synchronous designs. This application note explained the standards that define time sensitive networks and how the Precision Time Protocol enables clock synchronization. Then, it explained the hardware features and how to use them. Finally, it described the software needed to create a simple synchronized device using the LAN8650/1 as the basis of a time-aware 10BASE-T1S multidrop mixing segment.

6. References

- *IEEE Standard for Ethernet*, IEEE Std 802.3™-2022.
standards.ieee.org/standard/802_3-2022.html
- *IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*, IEEE Std 1588™-2019.
standards.ieee.org/standard/1588-2019.html
- *IEEE Standard for Local and Metropolitan Area Networks – Timing and Synchronization for Time-Sensitive Applications*, IEEE Std 802.1AS™-2020.
standards.ieee.org/standard/802_1AS-2020.html
- *10BASE-T1x MAC-PHY Serial Interface*, Version 1.1, OPEN Alliance, 2021.
www.opensig.org/Automotive-Ethernet-Specifications/
- *LAN8650/1 10BASE-T1S MAC-PHY Ethernet Controller with SPI Datasheet*, DS60001734, Microchip.
www.microchip.com/LAN8650
- *LAN8650/1 Configuration Application Note*, DS60001760, Microchip.
www.microchip.com/DS60001760
- *MPLAB® Harmony v3 LAN865x Driver Example*, DS00004964
www.microchip.com/LAN8650
- Ye, Weidong. "IEEE 1588 Clock servo algorithm." In *2009 9th International Conference on Electronic Measurement & Instruments*, pp. 1-341. IEEE, 2009.
- Correll, Kendall, Nick Barendt, and Michael Branicky. "Design considerations for software only implementations of the IEEE 1588 precision time protocol." In *Conference on IEEE*, vol. 1588, pp. 11-15. 2005..
- Time Synchronization sample application for LAN8650/1
github.com/MicrochipTech/LAN865x-TimeSync

7. Revision History

Table 7-1. Application Note Revision History

Revision Level & Date	Section/Figure/Entry	Changes
DS60001847C (June-2025)	Chapter 4	Clarified <i>Locked (Fine)</i> algorithm step according to demo code Added results section
	General Terms	Added One Pulse Per Second (1PPS)
	References	Added link to demo code
DS60001847B (Aug-2024)	All	Initial revision

Microchip Information

Trademarks

The “Microchip” name and logo, the “M” logo, and other names, logos, and brands are registered and unregistered trademarks of Microchip Technology Incorporated or its affiliates and/or subsidiaries in the United States and/or other countries (“Microchip Trademarks”). Information regarding Microchip Trademarks can be found at <https://www.microchip.com/en-us/about/legal-information/microchip-trademarks>.

ISBN: 979-8-3371-1342-5

Legal Notice

This publication and the information herein may be used only with Microchip products, including to design, test, and integrate Microchip products with your application. Use of this information in any other manner violates these terms. Information regarding device applications is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. Contact your local Microchip sales office for additional support or, obtain additional support at www.microchip.com/en-us/support/design-help/client-support-services.

THIS INFORMATION IS PROVIDED BY MICROCHIP “AS IS”. MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, AND FITNESS FOR A PARTICULAR PURPOSE, OR WARRANTIES RELATED TO ITS CONDITION, QUALITY, OR PERFORMANCE.

IN NO EVENT WILL MICROCHIP BE LIABLE FOR ANY INDIRECT, SPECIAL, PUNITIVE, INCIDENTAL, OR CONSEQUENTIAL LOSS, DAMAGE, COST, OR EXPENSE OF ANY KIND WHATSOEVER RELATED TO THE INFORMATION OR ITS USE, HOWEVER CAUSED, EVEN IF MICROCHIP HAS BEEN ADVISED OF THE POSSIBILITY OR THE DAMAGES ARE FORESEEABLE. TO THE FULLEST EXTENT ALLOWED BY LAW, MICROCHIP’S TOTAL LIABILITY ON ALL CLAIMS IN ANY WAY RELATED TO THE INFORMATION OR ITS USE WILL NOT EXCEED THE AMOUNT OF FEES, IF ANY, THAT YOU HAVE PAID DIRECTLY TO MICROCHIP FOR THE INFORMATION.

Use of Microchip devices in life support and/or safety applications is entirely at the buyer’s risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights unless otherwise stated.

Microchip Devices Code Protection Feature

Note the following details of the code protection feature on Microchip products:

- Microchip products meet the specifications contained in their particular Microchip Data Sheet.
- Microchip believes that its family of products is secure when used in the intended manner, within operating specifications, and under normal conditions.
- Microchip values and aggressively protects its intellectual property rights. Attempts to breach the code protection features of Microchip products are strictly prohibited and may violate the Digital Millennium Copyright Act.
- Neither Microchip nor any other semiconductor manufacturer can guarantee the security of its code. Code protection does not mean that we are guaranteeing the product is “unbreakable”. Code protection is constantly evolving. Microchip is committed to continuously improving the code protection features of our products.